CSC_51053_EP, Exercise Session: FDs and Concurrency Control

Ioana Manolescu, Madhulika Mohanty, Garima Gaur

November 21, 2025

The following comment applies to all exercises on FDs: *Most of the time, we work with dependencies having* ***a single attribute on the right-hand side****, because: this is the form in which 3NF is checked; and it is always possible to obtain such a cover from an arbitrary set of dependencies.*
We work with dependencies having several attributes on the right-hand side only when we need them for a specific axiom application step.

1 Keys, 3NF and BCNF [2]

Consider a relation $R(A, B, C, D, E)$ on which the following dependencies hold: $A \to B$, $BC \to E$, and $DE \to A$.

1. List all keys for R. C and D are determined by no other set of attributes (never appear in the right-hand side), hence they must be part of the key. It is not a key because $CD \not\to A$. We next show that ACD, CDB and CDE are the keys.
   $A \to B$ implies $AC \to BC$[1] then $AC \to E$ thus $AC \to BE$. Further, $DE \to A$ implies $CDE \to AC$ thus $CDE \to BE$, too, leading to $CDE \to ABCDE$ which entails CDE is a key.
   Further, $BC \to E$ thus $BCD \to DE$ and $DE \to A$, thus BCD is also a key.
   Further, $A \to B$ implies $AC \to BC$ and $BC \to E$ implies $AC \to E$, thus $ACD \to DE$ which together with $A \to B$ implies ACD is also a key.
   All the other supersets of CD are supersets of one of the key, hence we found all the keys.

2. Is R in third normal form (3NF)? Let F be the set of dependencies of a relation and F^+ its closure, i.e. the set of all dependencies implied by F. For a relation to be in 3NF, every F^+ dependency of the form $X \to A$, should either be (i) trivial ($A \in X$), or (ii) X is a superkey, or (iii) A is part of a key.
   In this example, any dependency of the form $X \to T$ (where T is one of the attributes of R, that is $T \in \{A, B, C, D, E\}$) will have T part of *a* key, because every attribute of R is part of at least one key! Thus, any dependency of the form $X \to T$ holding on R does not violate 3NF. Thus, R is in 3NF.

3. Is R in Boyce-Codd Normal Form (BCNF)? A relation is in BCNF if for any F^+ dependency of the form $X \to Y$ which holds on the relation, either $Y \subseteq X$, or X is a superkey.
   In this example, for instance, $A \to B$ yet A is not a superkey, thus the relation is not in BCNF.

2 Decomposition into BCNF [2]

You are given a relation $R(A, B, C, D, E, G, H)$ and a set of functional dependencies $F = \{AB \to C, AC \to B, AD \to E, B \to D, BC \to A, E \to G\}$.

1. For each of the following attribute sets, do the following. (i) Find a minimal cover of the set of dependencies that hold over the relation. (ii) What is the strongest normal form (3NF or BCNF) that a relation with these attributes satisfies? (iii) If that relation is not in BCNF, decompose it into BCNF.

   Call F the set of dependencies given over R. In each case, (i) requires finding for a given relation R_i that is a projection over R, the dependencies $(F^+)_{R_i}$ holding over R_i: these are the dependencies

[1] Observe how here we built a functional dependency with two attributes on the right in an opportunistic fashion, to enable inference based on $BC \to E$.

implied by F and such that their left-hand side and their right-hand side attributes appear in R_i.
Notation: While in the lecture slides we used the notation $(F)_{R_i}$, its meaning is defined based on the dependencies of F^+, and we use the notation $(F^+)_{R_i}$ here as it is clearer. Then, we have to find a minimal cover for $(F^+)_{R_i}$, that is: a minimal set of dependencies having, each, a single attribute on the right-hand side; and such that together, they imply all the dependencies of $(F^+)_{R_i}$.

(a) $CDEGH$ On this relation, the only functional dependency is $\{E \to G\}$ thus this is also the minimal cover. $CDEH$ is a key. The relation is not in 3NF because of $E \to G$: in this case, E is not a superkey, and G is not part of a key.
The relation cannot be BCNF since is not 3NF. We decompose R based on the dependency violating BCNF, thus we obtain $R_1(CDEH)$ and $R_2(EG)$.
Now, we consider $R_1(CDEH)$: to know if it is in BCNF, we must analyze its dependencies with respect to its keys. Let us first find dependencies that hold on $CDEH$. It turns out that $(F^+)_{R_1} = \emptyset$: no dependency that holds on R has both its left-hand side and its right-hand side in R_1.
If a relation has *only trivial functional dependencies*, it is in 3NF and in BCNF. Therefore, $R_1(CDEH)$ is in BCNF (and also 3NF).
For what concerns $R_2(EG)$, it has only the dependency $E \to G$, E is a key, and R_2 is in BCNF and 3NF.
Therefore, we have sufficiently decomposed $R(CDEGH)$ and we are finished with the BCNF decomposition.

(b) ABC The dependencies from F which hold over ABC are: $AB \to C, AC \to B, BC \to A$. In principle there could be some more dependencies in F^+ (but not in F) over the attributes ABC, but in this case there aren't; you may check this by developing (a minimal cover of) F^+. Thus, $(F^+)_{ABC} = \{AB \to C, AC \to B, BC \to A\}$.
Thus, ABC has three keys. With respect to 3NF, observe that every attribute is part of a key, thus all the functional dependencies respect 3NF and the relation is in 3NF. With respect to BCNF, all the dependencies have a (super-) key at the left, thus the relation is in BCNF.

(c) $ABCD$ The dependencies from F which hold over $ABCD$ are $AB \to C, AC \to B, BC \to A, B \to D$. Further, from $AC \to B$ and $B \to D$ it follows $AC \to D$, thus AC is a key. From $B \to D$ and $AB \to C$ it follows that AB is also a key. From $BC \to A$ and $B \to D$ it follows that BC is also a key.
It turns out that $AB \to C, AC \to B, BC \to A, B \to D$ is a minimal cover for $(F^+)_{ABCD}$.
With respect to 3NF: in $B \to D$, D is not part of a key, thus $ABCD$ is not in 3NF.
With respect to BCNF: in $B \to D$, B is not a superkey, thus $ABCD$ is not in BCNF. Picking $B \to D$, we decompose it into $R_1(BD)$ and $R_2(ABC)$. $R_1(BD)$ is in BCNF, it has the key B. $R_2(ABC)$ is the relation explored at point (a), it is in BCNF. Thus, the BCNF decomposition is finished.

(d) $ABCEG$ The dependencies from F which hold over $ABCEG$ are $AB \to C, AC \to B, BC \to A, E \to G$. Now note that from from $B \to D$ it follows $AB \to AD$ and given that $AD \to E$ it follows that $AB \to E$ is also a dependency from F^+ which holds over $ABCEG$. From $E \to G$ we also get $AB \to G$. Since we have $AB \to C$, $AB \to E$ and $AB \to G$, it is easy to see that AB is a key for $ABCEG$.
Further, from $AC \to B$ it follows $AC \to AB$ and given that AB is a key, clearly, AC is a key also for $ABCEG$.
Further, from $BC \to A$ it follows $BC \to AB$ and given that AB is a key, clearly, BC is a key also for $ABCEG$.
There is no other key for $ABCEG$. A minimal cover for this relation is $AB \to C, AC \to B, AB \to E, BC \to A, E \to G$.
3NF is violated by $E \to G$ since G is not part of any key.
BCNF is violated by $E \to G$ since E is not a superkey. We pick $E \to G$ and decompose into $R_1(ABCE)$, $R_2(EG)$. In R_1, there are three keys AB, AC, BC; any dependency holding on ABC has a superkey at the left, thus R_1 is in BCNF. In R_2, E is a key, and the relation is in BCNF. Thus, we are finished decomposing.

(e) $ACEH$ We have $AC \to B$ and $B \to D$ thus $AC \to D$ leading to $AC \to AD$, which together with $AD \to E$ leads to $AC \to E$. Thus, ACH is a key, and a minimal cover for $ACEH$ is $AC \to E$. Because E is not part of a key, $AC \to E$ violates 3NF; it also violates BCNF. We decompose $ACEH$ into ACE and ACH. ACE is in BCNF, having only the depen-

dency $AC \to E$; ACH is in BCNF, too, because it has no dependency. Thus, the BCNF decomposition is finished.

2. Only for the purpose of this point, assume the dependency $G \to H$ also holds. Redo the above analysis for $ACEH$. If $E \to G$ and $G \to H$, then $E \to H$; as shown above, we also have $AC \to E$, thus AC is a key. A minimal cover for $ACEH$' dependencies is $AC \to E, E \to H$. The relation is not in 3NF nor BCNF because of $E \to H$. We decompose it into ACE and EH, each of which is BCNF.

3. Which of the following decompositions of $R(ABCDEG)$, with the same set of dependencies F, are: dependency-preserving? lossless-join?

   A decomposition of R into R_1 and R_2 is dependency-preserving iff $((F^+)_{R_1} \cup (F^+)_{R_2})^+ = F^+$. A decomposition of R into R_1, R_2 is lossless-join iff for any instance, $R_1 \bowtie R_2 = R$. It is always the case that $R \subseteq (R_1 \bowtie R_2)$, however if the decomposition is lossless-join, there should be equality. A theorem states that the decomposition is lossless-join iff $R_1 \cap R_2 \to R_1$ or $R_1 \cap R_2 \to R_2$.

   (a) $\{AB, BC, ABDE, EG\}$ The FDs for $R(ABCDEG)$ are: $\{AB \to C, AC \to B, AD \to E, B \to D, BC \to A, E \to G\}$.
   With respect to **dependency preservation**, we have $(F^+)_{AB} = \emptyset$, $(F^+)_{BC} = \emptyset$. Thus, $(F_{AB} \cup F_{BC})^+ = \emptyset \neq \{AB \to C, BC \to A, AC \to B\}^+$ thus the decomposition is not dependency-preserving (more generally, we need to check that there is *no* relation R_i in the decomposition such that $AB \to C$ belongs to $(F^+)_{R_i}$; this is indeed the case here.).
   Wrt the **lossless-join** property, we need to establish if there exists a join expression of the form $AB \bowtie BC \bowtie ABDE \bowtie EG$ that is equivalent to $ABCDEG$.

   - Let us check for instance if it is possible to retrieve the ABC value combinations present in R, out of the four relations returned by the decomposition. These attributes are present in the tables AB, BC, and $ABDE$. Let us see for instance if (AB, BC) is a lossless join decomposition for ABC.
     Consider the split of ABC in AB and BC. We have $AB \cap BC = B$ and we do not have $B \to AB$ nor $B \to BC$. Thus, for some instances, $AB \bowtie BC \neq ABC$[2].
   - One still has to check whether joining $AB \bowtie BC$ with other tables resulting from the decomposition may enable to prune the erroneous tuples from $AB \bowtie BC$, to keep only the correct combinations of ABC values. In this example, this is not the case. Thus, the decomposition is not lossless-join.

   Another more direct proof is to *find a data instance* on which the decomposition is lossy-join. Consider an instance of R consisting of the tuples $\{(a_1, b_1, c_1, d_1, e_1, g_1), (a_2, b_1, c_2, d_1, e_1, g_1)\}$. However, $AB \bowtie BC \bowtie ABDE \bowtie EG$ contains $(a_1, b_1, c_2, d_1, e_1, g_1)$.

   (b) $\{ABC, ACDE, ADG\}$ We start with the analysis for **dependency-preservation**. The dependencies $AB \to C$, $AC \to B$ and $BC \to A$ are preserved because attributes ABC are together (formally: a minimal cover of $(F^+)_{ABC}$ is $\{AB \to C, AC \to B, BC \to A\}$ and all these dependencies have the left- and right-hand attributes in ABC, thus, they are preserved when ABC is separated from the others). $AD \to E$ is preserved in $ACDE$, however $B \to D$ is not preserved by ABC and $ACDE$; similarly, $E \to G$ is not preserved. Thus, this decomposition does not preserve dependencies.
   We now consider the **lossless-join** property. The decomposition is lossless-join if and only if $ABC \bowtie ACDE \bowtie ADG = ABCDEG$. One needs to pick an order for the joins, and we can pick e.g. $(ABC \bowtie ACDE) \bowtie ADG$.

   - Let's see if the split of $ABCDE$ into ABC and $ACDE$ is lossless-join. We have $ABC \cap ACDE = AC$, and we have $AC \to ABC$, thus this decomposition is OK.

   - Now let's look at the decomposition of $ABCDEG$ into $ABCDE$ (which as we have seen, we can obtain from the join above) and ADG. We have $ABCDE \cap ADG = AD$. Further, $AD \to E$ and $E \to G$ thus $AD \to G$, thus $AD \to ADG$ and therefore this decomposition is OK, too.

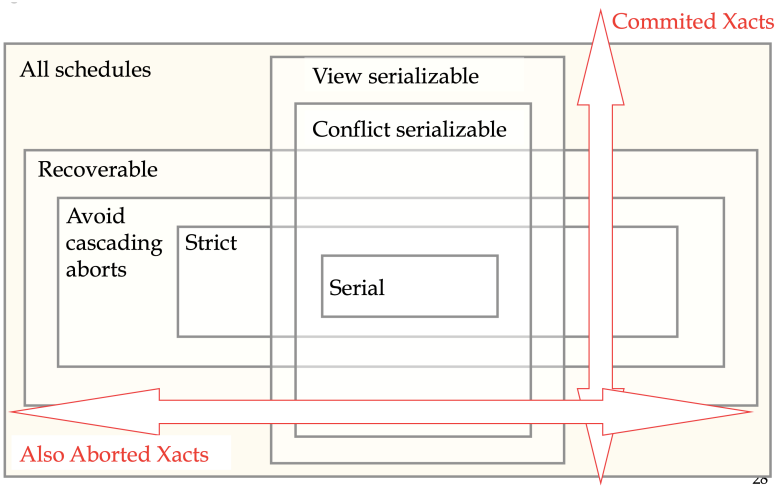
[2] Example: decomposing the instance $R^*_{ABC} = \{(a_1, b_1, c_1), (a_2, b_1, c_2)\}$ into AB and BC leads to $R^*_{AB} = \{(a_1, b_1), (a_2, b_1)\}$ and $R^*_{BC} = \{(b_1, c_1), (b_2, c_2)\}$. We see that $R^*_{AB} \bowtie R^*_{BC} = \{(a_1, b_1, c_1), (a_1, b_1, c_2), (a_2, b_1, c_1), (a_2, b_1, c_2)\} \neq R^*_{ABC}$.

Thus, this decomposition is lossless-join.

3 Schedule properties

3.1 Classes of Schedules [2]

Consider the following classes of schedules: *serializable, conflict-serializable, view-serializable, recoverable, avoids-cascading-aborts,* and *strict*. For each of the following schedules, state which of the preceding classes it belongs to (recall the inclusion diagram below):



The actions are listed in the order they are scheduled and prefixed with the transaction name.

1. T1:R(X), T2:R(X), T1:W(X), T2:W(X), T2:Commit, T1: Commit
2. T1:W(X), T2:R(Y), T1:R(Y), T2:R(X), T1: Commit, T2: Commit
3. T1:R(X), T2:R(Y), T3:W(X), T2:R(X), T1:R(Y): T1:Commit, T2:Commit, T3:Commit
4. T1:R(X), T2:R(Y), T3:W(X), T2:R(X), T1:R(Y): T1:Commit, T3:Commit, T2:Commit
5. T1:R(X), T2:W(X), T1:W(X), T2:Commit, T1:Commit
6. T1:R(X), T2:W(X), T1:W(Y), T2:R(Y), T2:Commit, T1:Commit

1. Not serializable: check possible serialization. Not strict: T2:W(X) done on X modified by uncommited T1. Avoids cascading aborts.
2. Conflict serializable (T1→T2), recoverable as T2 commits after T1, does not avoid cascading as T2 reads from T1 before T1 has commited
3. Conflict serializable (T1→T3→ T2), not recoverable
4. Conflict serializable, recoverable (T2 commits after T3), does not avoid cascading aborts.
5. Not serializable, not strict, avoids cascading aborts.
6. Conflict serializable, not recoverable.

3.2 More on Schedules [1]

Consider the schedule S shown below (operations are shown in temporal order, from left to right; we number the writes and reads to facilitate refering to them):

T_1			$R_1(A)$		$W_1(B)$				
T_2	$R_2(B)$	$W_2(A)$				$W_2(B)$			
T_3				$R_3(A)$			$W_3(B)$		

Answer the questions below providing a brief justification for your answer to each of them.

1. Is this schedule view serializable? If it is, propose a view-equivalent serial schedule. It's view serializable, equivalent to T_2,T_1,T_3

2. Is this schedule conflict serializable? If it is, propose a conflict-equivalent serial schedule. It's not conflict serializable (cycle between T_1 and T_2).
   Now consider the schedule S_2 where we also show transaction commits:

T_1			$R_1(A)$		$W_1(B)$					C_1		
T_2	$R_2(B)$	$W_2(A)$				$W_2(B)$			C_2			
T_3				$R_3(A)$			$W_3(B)$				C_3	

3. Is S_2 recoverable? Recoverable: T_2 reads from nobody, T_1 reads from T_2 but commits after T_2, T_3 commits last.

4. Does S_2 avoid cascading aborts? It does not avoid cascading aborts because T_1 reads from T_2 before T_2 commits.

5. Is S_2 strict? Not strict since it does not avoid cascading aborts.

References

[1] Hector Garcia-Molina, Jeffrey D. Ullman, and Jennifer Widom. *Database systems - the complete book (2. ed.)*. Pearson Education, 2009.

[2] Raghu Ramakrishnan and Johannes Gehrke. *Database management systems (3rd ed.)*. McGraw-Hill, 2003.